Driver Behaviour Recognition from Smartphone Motion Sensors: A Comparative Study of SVM, Random Forest and Bidirectional LSTM Classifiers

Harshit Girish Seth
Dept. of Artificial Intelligence & Data Science
MIT World Peace University
Pune, India
harshit.seth@mitwpu.edu.in

***Abstract*—Aggressive driving manoeuvres are a major contributor to road accidents, and their automatic recognition underpins usage-based insurance, fleet management and driver-coaching applications. This paper treats driving-event recognition as a multi-class time-series pattern recognition problem and compares three solutions on a public smartphone-sensor dataset of 69 video-labelled events in seven classes (non-aggressive, aggressive acceleration, braking, left/right turn and left/right lane change). Signals from the accelerometer, linear accelerometer, gyroscope and magnetometer are synchronised at 50 Hz and segmented into 2-s sliding windows. From 120 hand-crafted statistical features, Solution 1 selects the most informative ones by Mutual Information and classifies them with a support vector machine (RBF kernel). Solution 2 is a random forest with embedded Gini-importance feature selection. Solution 3 is a two-layer bidirectional LSTM that learns directly from the raw 12-channel sequences. To avoid the optimistic bias caused by overlapping windows, all models are evaluated with event-grouped, stratified 4-fold cross-validation repeated three times, with nested hyper-parameter tuning. The random forest achieves the highest accuracy (0.894), macro-F1 (0.821) and ROC-AUC (0.987). The MI-selected SVM (accuracy 0.853, macro-F1 0.810) has the lowest inference cost, and the BiLSTM reaches a macro-F1 of 0.815 without any feature engineering. Mutual Information selection shows that about half of the features (k = 60) suffice. Paired Wilcoxon tests show that the macro-F1 differences are not statistically significant at this data scale. Gyroscope yaw-rate features are the most discriminative, and 1.5–2 s windows give the best trade-off between context and specificity.**

***Index Terms*—driver behaviour profiling, pattern recognition, time-series classification, smartphone sensors, support vector machine, random forest, LSTM, grouped cross-validation.**

I. Introduction

Road traffic crashes kill about 1.19 million people every year and are the leading cause of death for people aged 5–29 [1]. Driver behaviour (harsh braking, rapid acceleration, sharp cornering and abrupt lane changes) is among the most controllable risk factors. Recognising such events automatically lets insurers price usage-based policies, lets fleet operators coach drivers, and lets safety applications warn drivers in real time.

Dedicated telematics hardware is costly, whereas almost every driver already carries a smartphone with an accelerometer, gyroscope and magnetometer. Each manoeuvre leaves a characteristic signature in these signals: braking and acceleration appear as longitudinal force, turns as sustained yaw rotation and lane changes as an S-shaped lateral pattern. Recognising the manoeuvre from a short window of multi-channel sensor data is therefore a classic *pattern recognition* problem with three stages: representation (features), a decision rule (classifier) and evaluation.

Because labelled driving data are expensive to collect, a practical question is which family of classifiers works best when data are scarce. Classical classifiers on hand-crafted features compete here with deep sequence models that learn their own representation. This study answers that question on a public benchmark. Its contributions are:

- a reproducible pipeline that synchronises four smartphone sensors and segments labelled events into fixed-length windows;
- implementation and comparison of three solutions: Mutual Information feature selection with an RBF-kernel SVM, a random forest with embedded feature selection, and a bidirectional LSTM;
- a leakage-free evaluation protocol using event-grouped, stratified, repeated cross-validation with nested tuning and paired significance testing;
- analyses of per-class behaviour, feature importance, window length and computational cost that guide model choice.

II. Related Work

Johnson and Trivedi [2] showed that a smartphone mounted in the vehicle can detect aggressive manoeuvres by fusing accelerometer, gyroscope and magnetometer data and matching events against templates with dynamic time warping. Eren *et al.* [3] likewise estimated driving behaviour from smartphone sensors using DTW and a Bayesian classifier to label drivers as safe or risky.

Ferreira Júnior *et al.* [4] released the dataset used here and systematically compared four classifiers (artificial neural network, SVM, random forest and Bayesian network) over many sensor combinations and window sizes. They found the gyroscope and accelerometer to be the most informative sensors. Carvalho *et al.* [5] then applied recurrent neural

networks (simple RNN, LSTM and GRU) to the same data, which motivates the deep sequence model in our study.

Two methodological points shape our design. First, consecutive sliding windows from one event are nearly identical. If they are split randomly between training and test sets, performance is overestimated, a well-known pitfall in sensor-based activity recognition. Second, tuning hyper-parameters on the test data biases error estimates [6]. We therefore group windows by event and nest tuning inside each training fold. Following Demšar [7], we compare classifiers with a non-parametric paired test rather than raw mean differences.

III. Problem Statement and Objectives

Let $\mathbf{x} \in \mathbb{R}^{T \times C}$ be a window of T time steps from $C = 12$ sensor channels (three axes each of accelerometer, linear acceleration, gyroscope and magnetometer, in the earth frame). The goal is a classifier $f: \mathbb{R}^{T \times C} \to \{1, \ldots, 7\}$ that maps each window to one of seven driving events. The objectives are to:

- build a clean, synchronised and windowed dataset from raw smartphone logs;
- implement three methodologically different solutions: a margin-based kernel classifier, a tree ensemble and a recurrent neural network;
- compare them on accuracy, macro-averaged precision, recall, F1 and ROC-AUC, per-class performance, robustness to window length and computational cost;
- test whether observed differences are statistically significant and identify the most discriminative sensor features.

IV. Dataset and Pre-processing

A. Dataset

The Driver Behavior Dataset [4] was recorded in a 2011 Honda Civic with a Motorola XT1058 smartphone fixed to the windshield. Two experienced drivers completed four trips of about 13 minutes each in dry, sunny conditions and performed the manoeuvres on a predefined schedule. A passenger filmed each trip with an on-screen millisecond timer, and every event was later labelled from the video with its start and end times. Table I lists the class distribution. The lane-change classes are notably small because 11 lane-change events were discarded during collection.

TABLE I
Class Distribution

Class	Events	2-s windows
Non-aggressive	14	100
Aggr. acceleration	12	88
Aggr. braking	12	27
Aggr. left turn	11	74
Aggr. right turn	11	75
Aggr. left lane chg.	4	6
Aggr. right lane chg.	5	8
Total	**69**	**378**

B. Synchronisation

The motion sensors sample at about 51 Hz and the magnetometer at about 102 Hz, each with its own timestamps. All twelve channels are linearly interpolated onto a common 50 Hz grid that starts at the trip's first collection instant, so the ground-truth event times (in seconds from trip start) map directly to sample indices.

C. Segmentation

A sliding window of length W (default 2 s, i.e. 100 samples) moves through each labelled event with a 0.25 s hop. An event shorter than W contributes a single window centred on it. This yields 378 windows. Each window inherits its event's label and an event identifier used for grouped validation. Fig. 1 shows the mean signature of each class. Turns are dominated by a sustained gyroscope-z (yaw) excursion whose sign gives the direction. Lane changes show an S-shaped lateral and yaw pattern, and braking shows a longitudinal deceleration. Non-aggressive driving and aggressive acceleration have little rotational content, which foreshadows the main confusion between them.

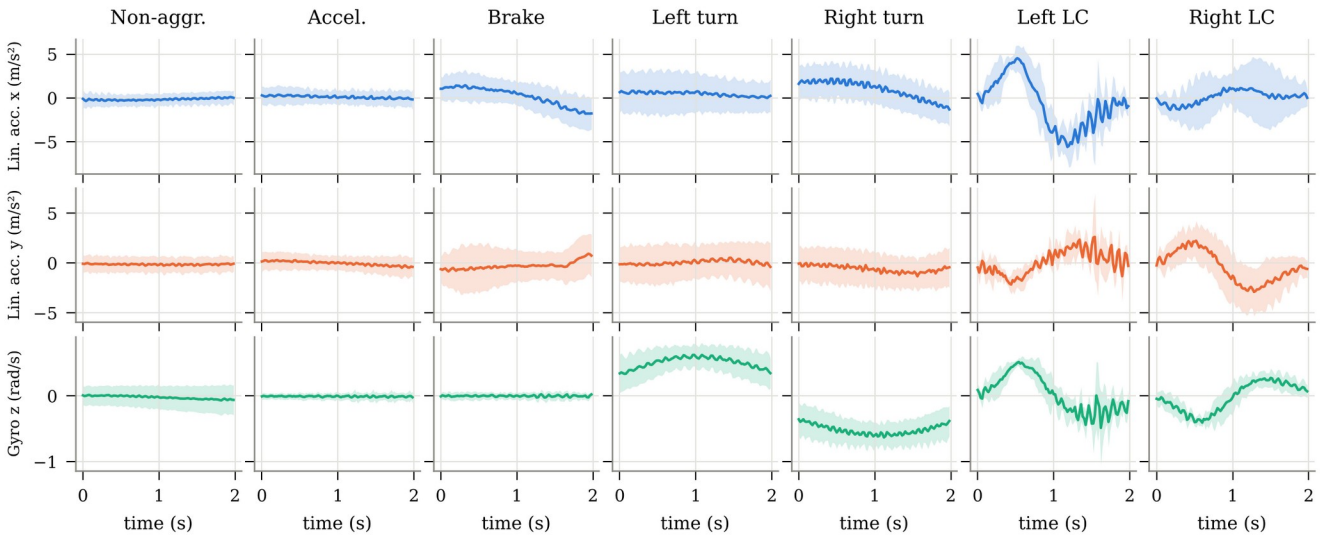


Fig. 1. Mean ± 1 SD of linear acceleration (x, y) and gyroscope z for every class over 2-s windows. Turns and lane changes are separable from the yaw channel alone; acceleration, braking and non-aggressive events differ mainly in longitudinal force.

V. Methodology

A. Feature Extraction (Solutions 1 and 2)

For each of the 12 channels, ten statistics are computed over the window: mean, standard deviation, minimum, maximum, median, range, skewness, kurtosis, root-mean-square and the slope of a least-squares linear fit. Together they capture the level, spread, shape and trend of each signal, giving a 120-dimensional feature vector. Fig. 2 projects these features with t-SNE [8]. Turns and braking form compact clusters, whereas non-aggressive and acceleration windows overlap.

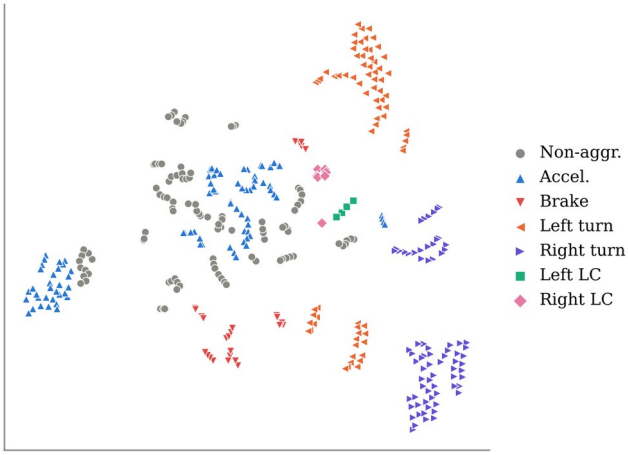


Fig. 2. t-SNE projection of the 120-D hand-crafted feature space, coloured by class.

B. Solution 1: Mutual Information Feature Selection + SVM

Not all 120 features are useful, and irrelevant ones add noise to a distance-based classifier. Solution 1 therefore applies a *filter* feature-selection step [16], [17] before classification. Each feature X is scored by its mutual information with the class label Y,

$$I(X;Y) = H(Y) - H(Y \mid X) \tag{1}$$

which measures how much knowing the feature reduces uncertainty about the class and, unlike correlation, captures non-linear dependence. MI between continuous features and the discrete label is estimated with the nearest-neighbour method of Ross [18]. The k highest-scoring features are retained. Fig. 3 shows the MI ranking and how SVM performance varies with k.

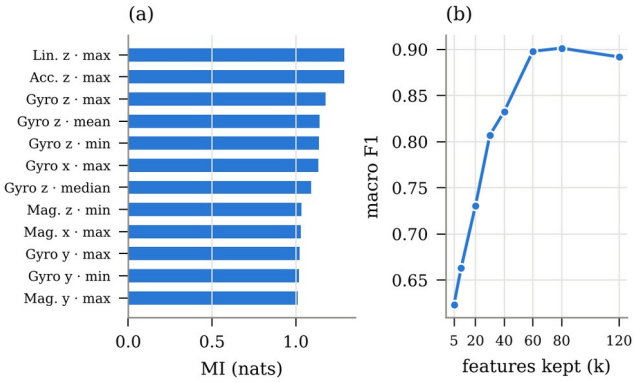


Fig. 3. Mutual Information feature selection: (a) the 12 features with the highest MI with the class label; (b) macro-F1 of the SVM as a function of the number of selected features k (4-fold event-grouped CV).

The selected features are classified by a support vector machine [9], which finds the hyperplane that maximises the margin between classes. With the radial basis function (RBF) kernel

$$K(\mathbf{x}, \mathbf{x}') = \exp(-\gamma \|\mathbf{x} - \mathbf{x}'\|^2) \tag{2}$$

it learns non-linear boundaries in an implicit high-dimensional space. Features are standardised before selection and classification because the kernel is distance-based. Multi-class decisions use the one-vs-one scheme, class weights compensate for imbalance, and Platt scaling provides probabilities for ROC analysis. The number of features $k \in \{20, 40, 60, 120\}$, $C \in$

{10, 100} and γ ∈ {scale, 10^{-3}} are tuned jointly by an inner event-grouped 3-fold grid search on each training fold, so MI scores are computed on training data only. The inner search most often kept k = 60 features (10 of 12 splits).

C. Solution 2: Random Forest with Embedded Feature Selection

A random forest [10] is an ensemble of B decision trees. Each tree is grown on a bootstrap sample and, at every split, considers only a random subset of $\sqrt{d}$ features. Averaging the decorrelated trees reduces variance without increasing bias. The predicted class probability is

$$\hat{p}(k \mid \mathbf{x}) = (1/B)\, \Sigma_b\, p_b(k \mid \mathbf{x}) \qquad (3)$$

Solution 2 uses an *embedded* feature-selection method, in which selection is part of model training [17]. A preliminary forest of 200 trees is fitted on all 120 features and ranks them by mean decrease in Gini impurity. Features below the median importance are discarded, and the final forest of B = 500 trees with balanced class weights is retrained on the remaining 60 features. Selection is repeated inside every training fold. No scaling is required.

D. Solution 3: Bidirectional LSTM

The LSTM [11] processes a sequence step by step and uses input, forget and output gates to retain relevant history while avoiding vanishing gradients. A bidirectional network [12] runs one LSTM forwards and one backwards and concatenates their states, so every time step sees the whole window. This suits offline event classification. Our model takes the z-scored 12-channel window down-sampled to 25 Hz (50 steps) and passes it through two stacked BiLSTM layers (64 units per direction, inter-layer dropout 0.3). It then mean-pools over time and ends in a dropout-regularised linear softmax layer over the seven classes.

The network is trained for 40 epochs with Adam (learning rate 2×10^{-3}, weight decay 10^{-4}), batch size 32 and class-weighted cross-entropy. Only ~280 windows are available per training fold, so data augmentation is applied on the fly: random per-channel scaling (σ = 0.1) and additive Gaussian jitter (σ = 0.05), both common for wearable-sensor data [13]. Normalisation statistics are computed on the training fold only, and no early stopping is used, so the test fold never influences training.

E. Evaluation Protocol

All windows from one event are assigned to the same fold with stratified group k-fold cross-validation (k = 4), and the procedure is repeated with three random seeds, giving 12 train/test splits. All three solutions use identical splits, so per-split scores are paired. We report accuracy and macro-averaged precision, recall and F1, which weight every class equally regardless of its size:

$$F1_{macro} = (1/K)\, \Sigma_k\, 2P_kR_k / (P_k + R_k) \qquad (4)$$

together with the macro one-vs-rest ROC-AUC, training time and per-window inference latency. The macro-F1 scores of each pair of models are compared with the two-sided Wilcoxon signed-rank test (α = 0.05) [7]. The pipeline is implemented in Python with scikit-learn [14] and PyTorch [15] and runs on a CPU.

VI. Results and Discussion

A. Overall Performance

Table II and Fig. 4 summarise the 12-split results. All three solutions perform far above the 14% chance level. The random forest gives the highest accuracy (0.894), macro-F1 (0.821) and ROC-AUC (0.987), and the lowest spread in accuracy across splits. The BiLSTM is close behind on macro-F1 (0.815) and has the highest macro recall while using no engineered features at all. The MI-selected SVM is second in accuracy (0.853) with a macro-F1 of 0.810. The three macro-F1 scores lie within about 0.01 of each other.

TABLE II
Classification Performance (Mean ± SD over 12 Splits)

Model	Accuracy	Macro-P	Macro-R	Macro-F1	AUC
SVM	0.853 ± 0.088	0.833 ± 0.110	0.825 ± 0.109	0.810 ± 0.119	0.983
Random Forest	0.894 ± 0.050	0.833 ± 0.115	0.853 ± 0.091	0.821 ± 0.111	0.987
BiLSTM	0.834 ± 0.081	0.838 ± 0.090	0.860 ± 0.080	0.815 ± 0.094	0.972

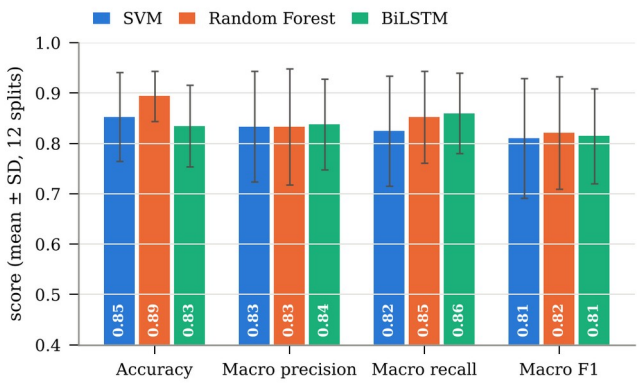


Fig. 4. Accuracy and macro-averaged precision, recall and F1 of the three solutions. Bars show the mean and whiskers ±1 SD over 12 event-grouped splits.

The gap between accuracy and macro-F1 is informative. Accuracy is dominated by the large classes, and the random forest excels there. Macro-F1 weights the four- and five-event lane-change classes as heavily as the rest. The relatively large standard deviations (≈0.1) are a direct consequence of the small number of events: a test fold may contain only one left-lane-change event, so one misclassified event moves the fold's macro-F1 substantially.

B. Class-Level Analysis

Fig. 5 shows the normalised confusion matrices pooled over the three repetitions. All models recognise turns very reliably, and the random forest classifies almost every turn window

correctly (F1 0.976 and 0.991). The most frequent error, shared by all three models, is between *non-aggressive* events and *aggressive acceleration*, since both lack lateral and rotational content and differ only in the magnitude of longitudinal force. Braking is occasionally mistaken for non-aggressive driving.

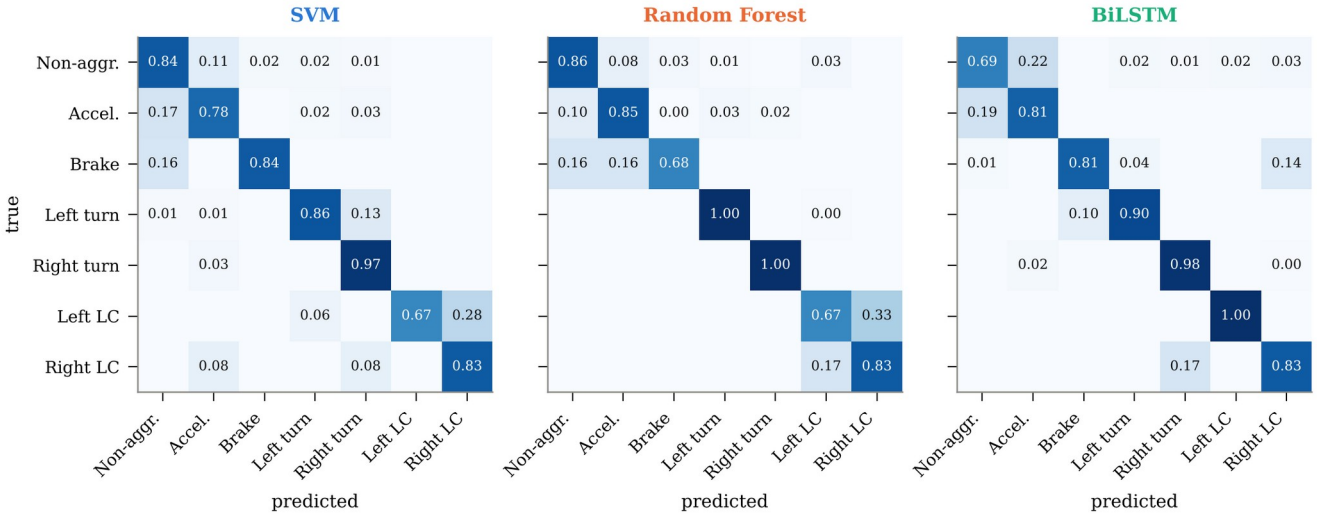


Fig. 5. Row-normalised confusion matrices (pooled over three repetitions of 4-fold event-grouped cross-validation). LC = lane change.

The models differ most on the rare lane-change classes (Fig. 6). The random forest confuses left with right lane changes (left-LC F1 = 0.568). With so few training examples, its bootstrap sampling and random feature subsets often miss the small sign difference that separates the two directions. The SVM, which uses all its support vectors rather than random subsets, is the most balanced on the two lane-change classes (F1 0.792 and 0.814) and is best on right lane changes. The BiLSTM recognises the temporal S-shape best for left lane changes (F1 0.838) but sometimes mistakes a right lane change for a right turn.

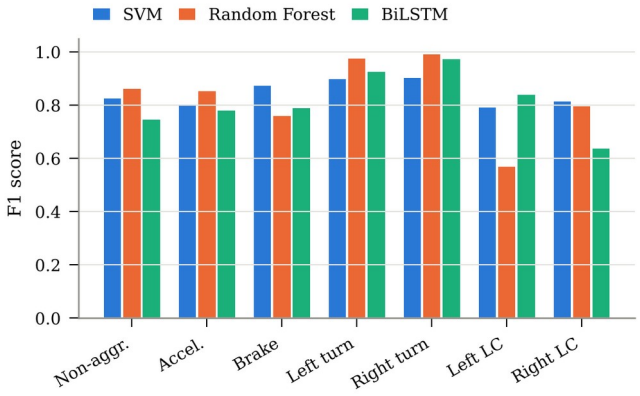


Fig. 6. Per-class F1 score (mean over three repetitions).

C. Feature Importance

Fig. 7 lists the most important features in the random forest. The gyroscope accounts for 40% of total importance, followed by linear acceleration (23%), the accelerometer (23%) and the magnetometer (13%). The four top-ranked features all describe gyroscope z (yaw rate), which separates left and right turns by sign and turns from straight-line events by magnitude. This agrees with the sensor ranking of [4] and suggests that a gyroscope-plus-accelerometer configuration would lose little accuracy while saving power.

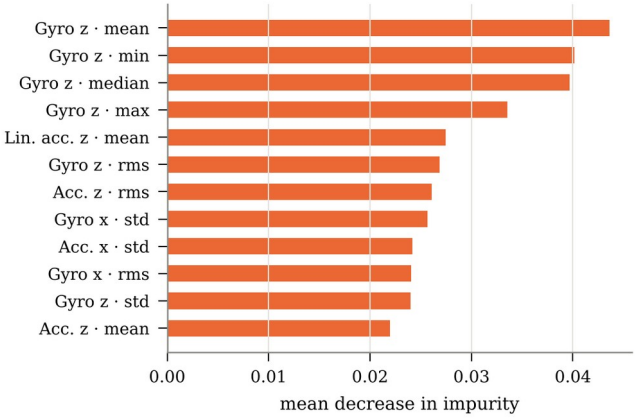


Fig. 7. Twelve most important features of the random forest (mean decrease in Gini impurity, averaged over 12 splits).

D. BiLSTM Training Behaviour

The learning curves in Fig. 8 show training loss falling steadily. Held-out loss drops quickly, levels off after about 10 epochs and does not rise again, while held-out accuracy keeps improving slowly. The persistent train/held-out gap reflects the small dataset, and augmentation and dropout keep it from growing into over-fitting. The held-out curve is shown for diagnosis only and was not used to select the epoch.

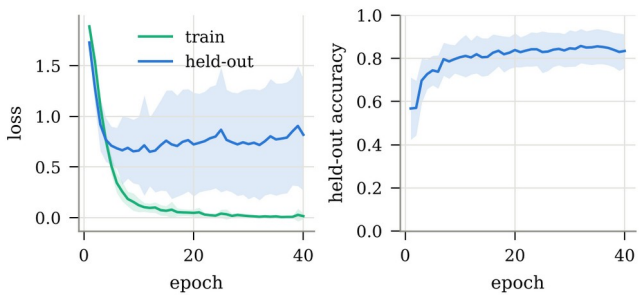


Fig. 8. BiLSTM training and held-out loss and held-out accuracy (mean ± SD over 12 splits).

E. Effect of Window Length

Fig. 9 varies the window length from 1 to 3 s (one repetition of 4-fold grouped CV). The SVM and BiLSTM both peak at 1.5 s (macro-F1 0.907 and 0.902) and degrade at both extremes. A 1-s window captures only part of a manoeuvre, whereas a 3-s window mixes in surrounding driving and yields fewer training windows. The random forest is the most robust, staying within 0.05 macro-F1 across all lengths. Windows of 1.5–2 s, about half the median event duration, are therefore recommended.

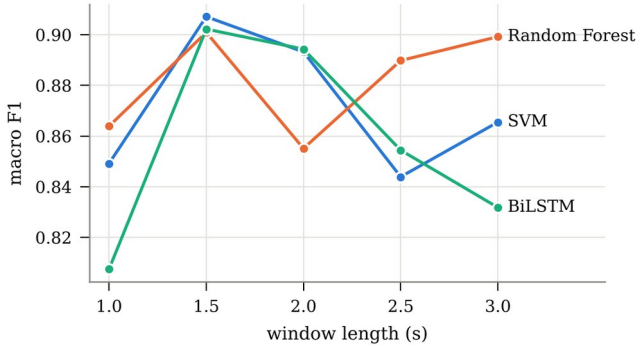


Fig. 9. Macro-F1 as a function of window length.

F. Computational Cost and Statistical Significance

Table III compares computational cost. The random forest, including its embedded selection step, trains in 1.45 s per split. The SVM is the slowest to train (14.63 s) because its nested grid search re-estimates Mutual Information for every candidate k and inner fold, which is a one-off offline cost. The BiLSTM needs 9.45 s on a CPU. All models classify a window in under 3 ms, comfortably real-time for a 50 Hz stream. The SVM is the cheapest at inference (0.023 ms), and the random forest's latency is dominated by traversing its 500 trees.

TABLE III
COMPUTATIONAL COST (CPU)

Model	Train / split (s)	Inference (ms/window)	Features
SVM	14.63	0.023	120 hand-crafted
Random Forest	1.45	2.181	120 hand-crafted
BiLSTM	9.45	0.158	none (raw signal)

Table IV reports the paired Wilcoxon tests on per-split macro-F1. None of the three pairwise differences is significant (all $p > 0.85$). On a dataset of 69 events, the three solutions are therefore statistically indistinguishable in macro-F1. The choice between them should rest on secondary criteria: interpretability and stability (random forest), latency and rare-class handling (SVM), or freedom from feature engineering and scalability to larger datasets (BiLSTM).

TABLE IV
PAIRED WILCOXON SIGNED-RANK TESTS ON MACRO-F1

Comparison (A vs. B)	Mean ΔF1	A / tie / B wins	p-value
Random Forest vs. SVM	+0.011	6 / 0 / 6	0.850
Random Forest vs. BiLSTM	+0.006	7 / 0 / 5	0.850
SVM vs. BiLSTM	-0.005	7 / 0 / 5	1.000

G. Limitations

The dataset comes from one vehicle, one phone model, two drivers and dry roads, so generalisation to other cars, mounting positions and conditions remains untested. It contains only labelled event segments, which means a deployed system would also need to separate events from ordinary background driving. The lane-change classes are very small, so their per-class scores carry high uncertainty.

VII. CONCLUSION AND FUTURE WORK

This study framed smartphone-based driver behaviour recognition as a seven-class time-series pattern recognition problem and compared three solutions under a leakage-free, event-grouped evaluation. All three recognise driving events well (accuracy 0.834–0.894, macro-F1 0.810–0.821). The random forest with embedded feature selection is the most accurate and robust. Mutual Information selection lets the SVM work with half the features while giving the fastest inference and the most balanced lane-change recognition. The BiLSTM matches both without hand-crafted features. Their macro-F1 differences are not statistically significant at this data scale. Yaw rate from the gyroscope is the single most discriminative signal, and 1.5–2 s windows work best.

Future work will collect a larger dataset from Pune city traffic across several vehicles and phone placements, add a background (no-event) class for continuous detection, and explore orientation-independent features. It will also evaluate hybrid CNN-LSTM and transformer models, which should benefit most from more data, and deploy the best model on-device for real-time feedback.

ACKNOWLEDGMENT

The author thanks J. Ferreira Júnior and co-authors for making the Driver Behavior Dataset publicly available, and the faculty of the Department of AI & DS, MIT-WPU, for their guidance.

References

[1] World Health Organization, *Global Status Report on Road Safety 2023*. Geneva, Switzerland: WHO, 2023.

[2] D. A. Johnson and M. M. Trivedi, “Driving style recognition using a smartphone as a sensor platform,” in *Proc. 14th Int. IEEE Conf. Intelligent Transportation Systems (ITSC)*, 2011, pp. 1609–1615.

[3] H. Eren, S. Makinist, E. Akin, and A. Yilmaz, “Estimating driving behavior by a smartphone,” in *Proc. IEEE Intelligent Vehicles Symp.*, 2012, pp. 234–239.

[4] J. Ferreira Júnior, E. Carvalho, B. V. Ferreira, C. de Souza, Y. Suhara, A. Pentland, and G. Pessin, “Driver behavior profiling: An investigation with different smartphone sensors and machine learning,” *PLoS ONE*, vol. 12, no. 4, e0174959, 2017.

[5] E. Carvalho, B. V. Ferreira, J. Ferreira, C. de Souza, H. V. Carvalho, Y. Suhara, A. S. Pentland, and G. Pessin, “Exploiting the use of recurrent neural networks for driver behavior profiling,” in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2017, pp. 3016–3021.

[6] S. Varma and R. Simon, “Bias in error estimation when using cross-validation for model selection,” *BMC Bioinformatics*, vol. 7, art. 91, 2006.

[7] J. Demšar, “Statistical comparisons of classifiers over multiple data sets,” *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[8] L. van der Maaten and G. Hinton, “Visualizing data using t-SNE,” *J. Mach. Learn. Res.*, vol. 9, pp. 2579–2605, 2008.

[9] C. Cortes and V. Vapnik, “Support-vector networks,” *Machine Learning*, vol. 20, no. 3, pp. 273–297, 1995.

[10] L. Breiman, “Random forests,” *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[11] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, 1997.

[12] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” *IEEE Trans. Signal Process.*, vol. 45, no. 11, pp. 2673–2681, 1997.

[13] T. T. Um, F. M. J. Pfister, D. Pichler, S. Endo, M. Lang, S. Hirche, U. Fietzek, and D. Kulić, “Data augmentation of wearable sensor data for Parkinson’s disease monitoring using convolutional neural networks,” in *Proc. 19th ACM Int. Conf. Multimodal Interaction (ICMI)*, 2017, pp. 216–220.

[14] F. Pedregosa *et al.*, “Scikit-learn: Machine learning in Python,” *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[15] A. Paszke *et al.*, “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32*, 2019, pp. 8024–8035.

[16] R. Battiti, “Using mutual information for selecting features in supervised neural net learning,” *IEEE Trans. Neural Netw.*, vol. 5, no. 4, pp. 537–550, 1994.

[17] I. Guyon and A. Elisseeff, “An introduction to variable and feature selection,” *J. Mach. Learn. Res.*, vol. 3, pp. 1157–1182, 2003.

[18] B. C. Ross, “Mutual information between discrete and continuous data sets,” *PLoS ONE*, vol. 9, no. 2, e87357, 2014.